# Exercise 3: Materializations

## Objective

Understand the four dbt materialization types — view, table, incremental, and ephemeral — and learn when to use each one in a real project.

## Prerequisites

- Exercise 2 completed (models `src_listings`, `src_hosts`, `src_reviews`, `fct_listings` exist and `dbt run` passes)

---

## About Materializations

When dbt runs a model, it must decide **how to persist the result** in the database. This is called the **materialization**. dbt has four built-in types:

| Materialization | What dbt creates | When to use |
|---|---|---|
| `view` (default) | A SQL view — no data stored | Fast-changing data, lightweight transformations |
| `table` | A physical table — data is copied | Downstream models read it frequently |
| `incremental` | A table — only new/changed rows are appended | Large tables where a full rebuild is expensive |
| `ephemeral` | Nothing — inlined as a CTE | Intermediate logic used only once |

---

## Step 1: Confirm the current state

Right now all your models are views (the dbt default). Confirm this:

```
import duckdb

conn = duckdb.connect("airbnb.duckdb")
print(conn.execute("SHOW ALL TABLES").df())
conn.close()
```

You will see `table_type = VIEW` for your `src_*` and `fct_listings` models.

---

## Step 2: Change a model to `table`

`fct_listings` is queried frequently by dashboards. Materializing it as a table means the JOIN is computed once and stored — not re-executed on every query.

Add a config block at the top of `models/marts/fct_listings.sql`:

```
{{ config(materialized='table') }}

WITH listings AS (
    SELECT * FROM {{ ref('src_listings') }}
),
...
```

Run:

```
dbt run --select fct_listings
```

Check the object type in DuckDB:

```
import duckdb

conn = duckdb.connect("airbnb.duckdb")
print(conn.execute("SHOW ALL TABLES").df())
conn.close()
```

`fct_listings` is now a `BASE TABLE`, not a `VIEW`.

---

## Step 3: Set default materializations per folder

Instead of adding a config block to every model, set defaults in `dbt_project.yml`. Open it and update the `models` section:

```
models:
  airbnb:
    src:
      +materialized: view
    marts:
      +materialized: table
```

This means:

- Everything in `models/src/` → view
- Everything in `models/marts/` → table

The `+` prefix means "apply to this folder and all subfolders".

Run `dbt run` to apply the new defaults to all models.

---

## Step 4: Create an incremental model

`fct_reviews_daily` aggregates reviews by day. Every day new reviews arrive, but old ones never change. A full rebuild would reprocess all historical data — wasteful. An incremental model only processes rows that are new since the last run.

Create `models/marts/fct_reviews_daily.sql`:

```
{{ config(materialized='incremental') }}

WITH reviews AS (
    SELECT * FROM {{ ref('src_reviews') }}
),

daily AS (
    SELECT
        review_date,
        COUNT(*)                        AS total_reviews,
        ROUND(AVG(rating), 2)           AS avg_rating,
        COUNT(CASE WHEN rating = 5
              THEN 1 END)               AS five_star_count
    FROM reviews
    {% if is_incremental() %}
        WHERE review_date > (SELECT MAX(review_date) FROM {{ this }})
    {% endif %}
    GROUP BY review_date
)

SELECT * FROM daily
```

### How `is_incremental()` works

On the **first run**, `is_incremental()` is `FALSE` — clears and builds the full table.

On **subsequent runs**, `is_incremental()` is `TRUE` — only processes rows newer than the latest row already in the table. dbt uses `{{ this }}` to reference the existing table.

Run the first time (full build):

```
dbt run --select fct_reviews_daily
```

Then force a full rebuild explicitly:

```
dbt run --select fct_reviews_daily --full-refresh
```

Use `--full-refresh` when the model schema changes or when you need to reprocess historical data.

---

## Step 5: Simulate an incremental run

Add a new review directly to the source seed to simulate new data arriving:

Open `seeds/raw_reviews.csv` and append one row:

```
1003,New Reviewer,5,2025-04-01
```

Re-load the seeds:

```
dbt seed
```

Now run the incremental model:

```
dbt run --select fct_reviews_daily
```

Check what dbt compiled — open `target/compiled/airbnb/models/marts/fct_reviews_daily.sql`. You will see the `WHERE review_date > (SELECT MAX(...))` clause was included. Only the new date was processed and inserted.

Verify in DuckDB:

```
import duckdb

conn = duckdb.connect("airbnb.duckdb")
print(conn.execute("SELECT * FROM fct_reviews_daily ORDER BY review_date").df())
conn.close()
```

---

## Step 6: Create an ephemeral model

Ephemeral models are **never materialized** in the database. dbt inlines them as CTEs inside the models that reference them. Use them for intermediate transformation logic that is too complex for a single CTE but not useful to expose as a table.

Create `models/src/_int_listing_with_host.sql`:

```
{{ config(materialized='ephemeral') }}

WITH listings AS (
    SELECT * FROM {{ ref('src_listings') }}
),

hosts AS (
    SELECT * FROM {{ ref('src_hosts') }}
)

SELECT
    l.listing_id,
    l.listing_name,
    l.neighbourhood,
    l.room_type,
    l.price_usd,
    h.host_name,
    h.is_superhost
FROM listings l
LEFT JOIN hosts h ON l.host_id = h.host_id
```

Convention: prefix ephemeral models with `_int_` (intermediate) so other developers know they are not stored in the database.

Now simplify `fct_listings.sql` to use this ephemeral model:

```
{{ config(materialized='table') }}

WITH enriched AS (
    SELECT * FROM {{ ref('_int_listing_with_host') }}
)

SELECT
    listing_id,
    listing_name,
    neighbourhood,
    room_type,
    price_usd,
    host_name,
    is_superhost
FROM enriched
```

Run:

```
dbt run --select fct_listings
```

Check `target/compiled/airbnb/models/marts/fct_listings.sql`. You will see that `_int_listing_with_host` was **inlined as a CTE** — it does not appear in `SHOW ALL TABLES` because it was never created in the database.

---

## Summary

```
seeds (raw_*)
    │
    ▼
src_listings, src_hosts, src_reviews   ← VIEW (cheap, always up to date)
    │
    ├──► _int_listing_with_host        ← EPHEMERAL (inlined, never stored)
    │         │
    │         ▼
    │     fct_listings                 ← TABLE (pre-computed, fast reads)
    │
    └──► fct_reviews_daily             ← INCREMENTAL (append-only, cost-efficient)
```

---

## Questions

1. A model is queried 10,000 times per day. Which materialization is best?
2. A model processes a 1 TB event log and only 0.1% of rows are new each run. Which materialization saves the most compute cost?
3. You change a column name in an incremental model. Why must you run `--full-refresh`?
4. Can an ephemeral model be referenced by more than one downstream model? What is the trade-off?